

# Mathematical Foundations

ACRONYMS

|     |                           |
|-----|---------------------------|
| BST | Binary Search Tree        |
| CFG | Context-Free Grammar      |
| CSG | Context-Sensitive Grammar |
| FSM | Finite-State Machine      |
| GCD | Greatest Common Divisor   |
| IH  | Induction Hypothesis      |
| LHS | Left-Hand Side            |
| PSG | Phrase Structure Grammar  |
| RHS | Right-Hand Side           |

INTRODUCTION

Software engineers can write code only for something that follows well-understood, unambiguous logic. The Mathematical Foundations KA helps software engineers comprehend this logic, which they translate into source code. The mathematics in this KA differ greatly from typical arithmetic, which deals with numbers. This KA focuses on logic and reasoning, which are the essence of the mathematics a software engineer must address.

Mathematics, in a sense, is the study of formal systems. The word formal is associated with preciseness, so there can be no ambiguous or erroneous interpretation of the facts. Mathematics is therefore the study of all certain truths about any concept. This concept can be about numbers, symbols, images, sounds or video — almost anything. In short, numbers and numeric equations aren’t the only subjects

of preciseness. On the contrary, a software engineer must have a precise abstraction on complex, diverse application domains.

The Mathematical Foundations KA covers basic techniques to identify a set of rules for reasoning in the context of the system under study. Anything you can deduce following these rules is an absolute certainty within the context of that system. This KA defines and discusses techniques that can represent and take forward a software engineer’s reasoning and judgment in a precise (and therefore mathematical) manner. The language and methods of logic discussed allow software engineers to describe mathematical proofs to infer conclusively the absolute truth of certain concepts beyond just numbers. This KA’s objective is to help software engineers develop the skill to identify and describe such logic and verify that the logic in the code is consistent with abstractions. The emphasis is on helping software engineers understand the basic concepts rather than on developing their arithmetic abilities.

BREAKDOWN OF TOPICS FOR MATHEMATICAL FOUNDATIONS

The breakdown of topics for the Mathematical Foundations KA is shown in Figure 17.1.

1. Basic Logic [1\*, c1]

1.1. Propositional Logic

A proposition is a statement that is either true or false, but not both. Consider declarative sentences for which it is meaningful to assign either of the two status values: true

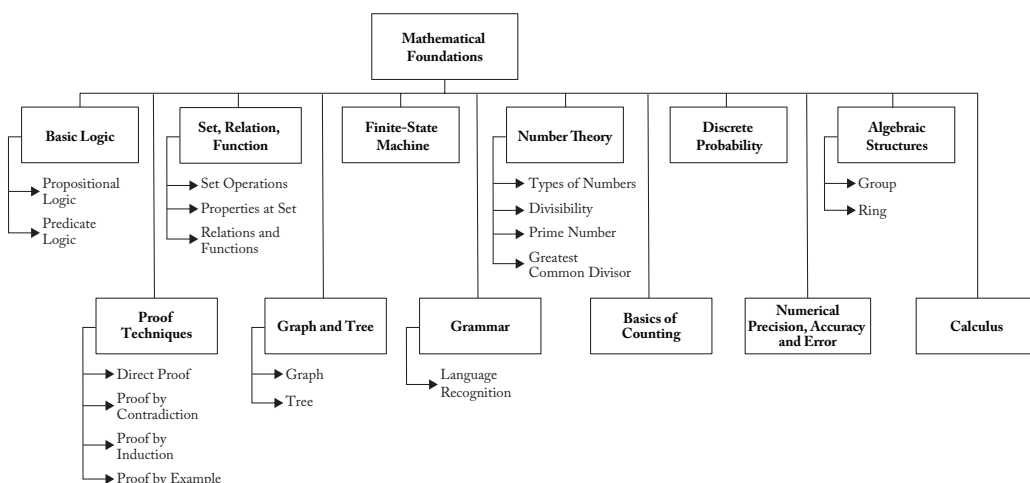


Figure 17.1. Breakdown of Topics for the Mathematical Foundations KA

or false. The following are some examples of propositions:

- The sun is a star.
- Elephants are mammals.
- $2 + 3 = 5$ .

However,  $a + 3 = b$  is not a proposition, as it is neither true nor false. Whether it is true depends on the values of the variables  $a$  and  $b$ .

*The Law of Excluded Middle:* For every proposition  $p$ , either  $p$  is true, or  $p$  is false.

*The Law of Contradiction:* For every proposition  $p$ , it is not the case that  $p$  is both true and false.

*Propositional logic* is the area of logic that deals with propositions. A *truth table* displays the relationships between the truth values of propositions.

A *Boolean variable* is a variable whose value is either true or false. Computer bit operations correspond to logical operations of Boolean variables.

The basic logical operators are negation (not,  $\neg p$ ), conjunction (and,  $p \wedge q$ ), disjunction (or,  $p \vee q$ ), exclusion ( $p \oplus q$ ), and implication ( $p \rightarrow q$ ). Compound propositions may be formed using various logical operators.

A compound proposition that is always true is a *tautology*. A compound proposition

that is always false is a *contradiction*. A compound proposition that is neither a tautology nor a contradiction is a *contingency*.

Compound propositions that always have the same truth value are called *logically equivalent* (denoted by  $\equiv$ ). Some common logical equivalences are the following:

- Identity laws:  

$$p \wedge T \equiv p \quad p \vee F \equiv p$$
- Domination laws:  

$$p \vee T \equiv T \quad p \wedge F \equiv F$$
- Idempotent laws:  

$$p \vee p \equiv p \quad p \wedge p \equiv p$$
- Double negation law:  

$$\neg(\neg p) \equiv p$$
- Commutative laws:  

$$p \vee q \equiv q \vee p \quad p \wedge q \equiv q \wedge p$$
- Associative laws:  

$$(p \vee q) \vee r \equiv p \vee (q \vee r)$$

$$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$$
- Distributive laws:  

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$$

$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

- De Morgan's laws:  
 $\neg (p \wedge q) \equiv \neg p \vee \neg q$   
 $\neg (p \vee q) \equiv \neg p \wedge \neg q$

### 1.2. Predicate Logic

A *predicate* is a verb phrase template that describes a property of objects or a relationship among objects represented by the variables. For example, in the sentence *The flower is red*, the template *is red* is a predicate. It describes a property of the flower. The same predicate may be used in other sentences.

Predicates are often given a name (e.g., *red* or simply *R*) that can represent the predicate (in this case, *red* or *R* can represent the predicate *is red*). Assuming *R* is the name for the predicate *is red*, sentences that assert an object has the color red can be represented as  $R(x)$ , where  $x$  represents an arbitrary object.  $R(x)$  reads as *x is red*.

Quantifiers allow statements about entire collections of objects so that enumerating each object by name is not necessary.

- The *universal quantifier*  $\forall x$  asserts that a sentence is true for all values of variable  $x$  (e.g.,  $\forall x \text{ Tiger}(x) \rightarrow \text{Mammal}(x)$  means all tigers are mammals).
- The *existential quantifier*  $\exists x$  asserts that a sentence is true for at least one value of variable  $x$  (e.g.,  $\exists x \text{ Tiger}(x) \rightarrow \text{Man-eater}(x)$  means there exists at least one tiger that is a man-eater).

Thus, while universal quantification uses implication, existential quantification naturally uses conjunction.

A variable  $x$  introduced into a logical expression by a quantifier is bound to the closest enclosing quantifier. Similarly, in a block-structured programming language, a variable in a logical expression refers to the closest quantifier within whose scope it appears. For example, in  $\exists x (\text{Cat}(x) \wedge \forall x (\text{Black}(x)))$ ,  $x$  in  $\text{Black}(x)$  is universally quantified. Therefore, the predicate implies that at least one cat exists and everything is black.

A variable is a *free variable* if it is not bound to a quantifier.

Propositional logic falls short in representing many assertions used in mathematics, computer science and, therefore, software engineering. It also fails to compare equivalence and other relationships between propositions. For example, the assertion “*a is greater than 1*” is not a proposition because one cannot infer whether it is true or false without knowing the value of *a*. Thus, propositional logic cannot deal with such sentences. However, such assertions appear quite often in mathematics, and we want to infer information from those assertions. Also, propositional logic cannot capture the pattern involved in the following two logical equivalences: “*Not all men are smokers*” and “*Some men don't smoke*.” Each of these two propositions is treated independently in propositional logic. There is no mechanism in propositional logic to determine whether the two are equivalent. Hence, propositional logic treats each equivalent proposition individually rather than apply a general formula that covers all equivalences collectively.

Predicate logic addresses these issues. In a sense, predicate logic (also known as first-order logic or predicate calculus) extends propositional logic to formulas involving terms and predicates.

## 2. Proof Techniques [1\*, c1]

A *proof* is an argument that rigorously establishes the truth of a statement. Proofs can themselves be represented formally as discrete structures.

Statements used in a proof include axioms and postulates that are essentially the underlying assumptions about mathematical structures, the hypotheses of the theorem to be proved and previously proved theorems.

- A *theorem* is a statement that can be shown to be true.
- A *lemma* is a simple theorem used in proving other theorems.
- A *corollary* is a proposition that can be established directly from a theorem that has been proved.

- A *conjecture* is a statement whose truth value is unknown.

When a conjecture's proof is found, that conjecture becomes a theorem. Many times, conjectures are shown to be false and, hence, are not theorems.

### 2.1. Direct Proof

Direct proof is a technique to establish that the implication  $p \rightarrow q$  is true by showing that  $q$  must be true when  $p$  is true. For example, to show that if  $n$  is odd, then  $n^2 - 1$  is even, suppose  $n$  is odd for some integer  $k$  — i.e.,  $n = 2k + 1$ :

$$\therefore n^2 = (2k + 1)^2 = 4k^2 + 4k + 1$$

As the first two terms of the Right-Hand Side (RHS) are even numbers irrespective of the value of  $k$ , the Left-Hand Side (LHS) ( $n^2$ ) is an odd number. Therefore,  $n^2 - 1$  is even. Direct proof can also be called *Proof by Deduction*.

### 2.2. Proof by Contradiction

A proposition  $p$  is true by contradiction if proved based on the truth of the implication  $\neg p \rightarrow q$ , where  $q$  is a contradiction. For example, to show that the sum of  $2x + 1$  and  $2y - 1$  is even, assume that the sum of  $2x + 1$  and  $2y - 1$  is odd. In other words,  $2(x + y)$ , which is a multiple of 2, is odd. This is a contradiction. Hence, the sum of  $2x + 1$  and  $2y - 1$  is even.

An inference rule is a pattern establishing that if a set of premises are all true, then it can be deduced that a certain conclusion statement is true. The reference rules of addition, simplification and conjunction need to be studied.

A closely related approach, *Proof by Contrapositive*, takes the opposite approach by assuming the conclusion is false and proving that the hypothesis is also false. If it can be shown that  $\neg q \rightarrow \neg p$  is true, then  $p \rightarrow q$  must also be true.

### 2.3. Proof by Induction

Proof by induction is done in two parts. First, the proposition is established to be true for a base case — typically for the positive integer 1. Then, in the second part, it is established that if the proposition holds for an arbitrary positive integer  $k$ , then it must also hold for the next greater integer,  $k + 1$ . In other words, proof by induction is based on the rule of inference that tells us that the truth of an infinite sequence of propositions  $P(n)$ ,  $\forall n \in [1 \dots \infty]$  is established if first  $P(1)$  is true, and, second  $\forall k \in [2 \dots n]$  if  $P(k) \rightarrow P(k + 1)$ .

For a proof by induction, it is not assumed that  $P(k)$  is true for all positive integers  $k$ . Proving a theorem or proposition only requires us to establish that if it is assumed  $P(k)$  is true for any arbitrary positive integer  $k$ , then  $P(k + 1)$  is also true. An in-depth discussion of the correctness of induction as a valid proof technique is beyond the scope of this KA. The following proposition is proved using induction:

Proposition: The sum of the first  $n$  positive odd integers  $S(n)$  is  $n^2$ .

Basis Step: The proposition is true for  $n = 1$  as  $S(1) = 1^2 = 1$ . The basis step is complete.

Inductive Step: The induction hypothesis (IH) is that the proposition is true for  $n = k$ ,  $k$  being an arbitrary positive integer.

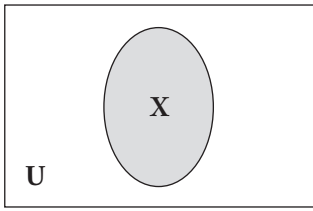
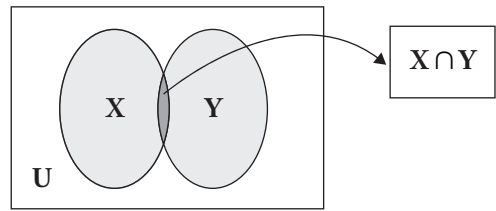
$$\therefore 1 + 3 + 5 + \dots + (2k - 1) = k^2$$

Now, it's to be shown that  $S(k) \rightarrow S(k + 1)$ .

$$\begin{aligned} S(k + 1) &= 1 + 3 + 5 + \dots + (2k - 1) + (2k + 1) \\ &= S(k) + (2k + 1) \\ &= k^2 + (2k + 1) \text{ [using IH]} \\ &= k^2 + 2k + 1 \\ &= (k + 1)^2 \end{aligned}$$

Thus, it is shown that if the proposition is true for  $n = k$ , then it is also true for  $n = k + 1$ .

The basis step together with the inductive step of the proof show that  $S(1)$  is true and the conditional statement  $S(k) \rightarrow S(k + 1)$  is true for all positive integers  $k$ . Hence, the proposition is proved.


 Figure 17.2. Venn Diagram for Set  $X$ 

 Figure 17.3. Intersection of Sets  $X$  and  $Y$ 

### 2.4. Proof by Example

Proof by example is only valid when the core of the proof is “there exists” and one needs only to show that at least one valid instance does exist. More generally, however, proof by example has often been called Inappropriate Generalization where validity is assumed to be illustrated through one or a few examples rather than a full proof. Showing only one or a few specific examples where  $p \rightarrow q$  is not sufficient to prove that for all cases  $p \rightarrow q$ .

### 3. Set, Relation, Function [1\*, c2]

**Set.** A set is a collection of objects called elements. A set can be represented by listing its elements between braces (e.g.,  $S = \{1, 2, 3\}$ ).

The symbol  $\in$  is used to express that an element belongs to a set or is a member of the set. Its negation is represented by  $\notin$  (e.g.,  $1 \in S$ , but  $4 \notin S$ ).

In a more compact representation of a set using set builder notation,  $\{x \mid P(x)\}$  is the set of all  $x$  such that  $P(x)$  for any proposition  $P(x)$  over any universe of discourse. Examples of important sets include the following:

- $N = \{0, 1, 2, 3, \dots\}$  = the set of nonnegative integers.
- $Z = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$  = the set of integers.

**Finite and Infinite Set.** A set with a finite number of elements is called a finite set. Conversely, any set that does not have a finite number of elements in it is an infinite set. For example, the set of all natural numbers is an infinite set.

**Cardinality.** The cardinality of a finite set  $S$  is the number of elements in  $S$ . This is represented as  $|S|$  (e.g., if  $S = \{5, 6, 7\}$ , then  $|S| = 3$ ).

**Universal Set.** In general,  $S = \{x \in U \mid p(x)\}$ , where  $U$  is the universe of discourse in which the predicate  $P(x)$  must be interpreted. The *universe of discourse* for a given predicate is often referred to as the *universal set*. Alternatively, one may define a universal set as the set of all elements.

**Set Equality.** Two sets are equal if and only if they have the same elements —

i.e.,  $X = Y \equiv \forall p (p \in X \leftrightarrow p \in Y)$ .

**Subset.**  $X$  is a subset of set  $Y$ , or  $X$  is contained in  $Y$ , if all elements of  $X$  are included in  $Y$ . This is denoted by  $X \subseteq Y$ . In other words,  $X \subseteq Y$  if and only if  $\forall p (p \in X \rightarrow p \in Y)$ . If  $X = \{1, 2, 3\}$  and  $Y = \{1, 2, 3, 4, 5\}$ , then  $X \subseteq Y$ . If  $X$  is not a subset of  $Y$ , it is denoted as  $X \not\subseteq Y$ .

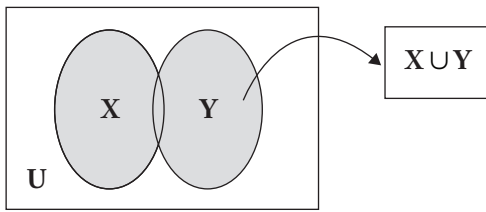
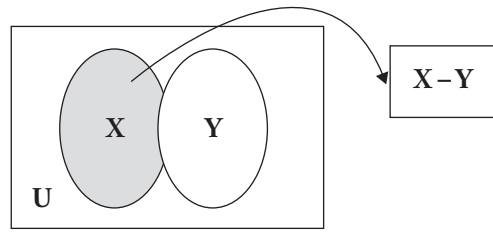
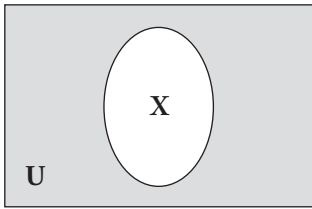
**Proper Subset.**  $X$  is a proper subset of  $Y$  (denoted by  $X \subset Y$ ) if  $X$  is a subset of  $Y$  but not equal to  $Y$  — i.e., there is some element in  $Y$  that is not in  $X$ .

In other words,  $X \subset Y$  if and only if  $(X \subseteq Y) \wedge (X \neq Y)$ . If  $X = \{1, 2, 3\}$ ,  $Y = \{1, 2, 3, 4\}$ , and  $Z = \{1, 2, 3\}$ , then  $X \subset Y$ , but  $X$  is not a proper subset of  $Z$ . Sets  $X$  and  $Z$  are equal sets.

If  $X$  is not a proper subset of  $Y$ , it is denoted as  $X \not\subset Y$ .

**Superset.** If  $X$  is a subset of  $Y$ , then  $Y$  is called a superset of  $X$ . This is denoted by  $Y \supseteq X$  — i.e.,  $Y \supseteq X$  if and only if  $X \subseteq Y$ . If  $X = \{1, 2, 3\}$  and  $Y = \{1, 2, 3, 4, 5\}$ , then  $Y \supseteq X$ .

**Empty Set.** A set with no elements is called an empty set. An empty set, denoted by  $\emptyset$ , is also referred to as a null or void set.

Figure 17.4. Union of Sets  $X$  and  $Y$ Figure 17.6. Venn Diagram for  $X - Y$ Figure 17.5. Venn Diagram for Complement Set of  $X$ 

**Power Set.** The set of all subsets of a set  $X$  is called the power set of  $X$ . It is represented as  $\wp(X)$ . If  $X = \{a, b, c\}$ , then  $\wp(X) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$ . If  $|X| = n$ , then  $|\wp(X)| = 2^n$ .

**Venn Diagrams.** Venn diagrams are graphic representations of sets as enclosed areas in the plane. For example, in Figure 17.2, the rectangle represents the universal set, and the shaded region represents a set  $X$ .

### 3.1. Set Operations

**Intersection.** The intersection of two sets,  $X$  and  $Y$ , denoted by  $X \cap Y$ , is the set of common elements in both  $X$  and  $Y$ . In other words,  $X \cap Y = \{p \mid (p \in X) \wedge (p \in Y)\}$ . For example,  $\{1, 2, 3\} \cap \{3, 4, 6\} = \{3\}$ .

If  $X \cap Y = \emptyset$ , then the two sets  $X$  and  $Y$  are said to be *disjoint*.

A Venn diagram for set intersection is shown in Figure 17.3. The common portion of the two sets represents the set intersection.

**Union.** The union of two sets,  $X$  and  $Y$ , denoted by  $X \cup Y$ , is the set of all elements in  $X$ , in  $Y$  or in both. In other words,  $X \cup Y = \{p \mid (p \in X) \vee (p \in Y)\}$ . For example,  $\{1, 2, 3\} \cup \{3, 4, 6\} = \{1, 2, 3, 4, 6\}$ .

It may be noted that  $|X \cup Y| = |X| + |Y| - |X \cap Y|$ .

A Venn diagram illustrating the union of two sets is represented by the shaded region in Figure 17.4.

**Complement Set.** The set of elements in the universal set that do not belong to a given set  $X$  is called its complement set  $X'$ . In other words,  $X' = \{p \mid (p \in U) \wedge (p \notin X)\}$ .

The shaded portion of the Venn diagram in Figure 17.5 represents the complement set of  $X$ .

**Set Difference or Relative Complement.** The set of elements that belong to set  $X$  but not to set  $Y$  builds the set difference of  $Y$  from  $X$ . This is represented by  $X - Y$ . In other words,  $X - Y = \{p \mid (p \in X) \wedge (p \notin Y)\}$ . For example,  $\{1, 2, 3\} - \{3, 4, 6\} = \{1, 2\}$ .

It can be proved that  $X - Y = X \cap Y'$ .

The set difference  $X - Y$  is illustrated by the shaded region in Figure 17.6 using a Venn diagram.

**Cartesian Product.** An ordinary pair  $\{p, q\}$  is a set with two elements. In a set, the order of the elements is irrelevant, so  $\{p, q\} = \{q, p\}$ .

In an ordered pair  $(p, q)$ , the order of occurrences of the elements is relevant. Thus,  $(p, q) \neq (q, p)$  unless  $p = q$ . In general,  $(p, q) = (s, t)$  if and only if  $p = s$  and  $q = t$ .

Given two sets,  $X$  and  $Y$ , their Cartesian product  $X \times Y$  is the set of all ordered pairs  $(p, q)$  such that  $p \in X$  and  $q \in Y$ . In other words,  $X \times Y = \{(p, q) \mid (p \in X) \wedge (q \in Y)\}$ . For example,  $\{a, b\} \times \{1, 2\} = \{(a, 1), (a, 2), (b, 1), (b, 2)\}$ .

### 3.2. Properties of Sets

Some of the important properties and laws of sets are:

Associative Laws:

- $X \cup (Y \cap Z) = (X \cup Y) \cap Z$
- $X \cap (Y \cup Z) = (X \cap Y) \cup Z$

Commutative Laws:

- $X \cup Y = Y \cup X$        $X \cap Y = Y \cap X$

Distributive Laws:

- $X \cup (Y \cap Z) = (X \cup Y) \cap (X \cup Z)$
- $X \cap (Y \cup Z) = (X \cap Y) \cup (X \cap Z)$

Identity Laws:

- $X \cup \phi = X$        $X \cap U = X$

Complement Laws:

- $X \cup X' = U$        $X \cap X' = \phi$

Idempotent Laws:

- $X \cup X = X$        $X \cap X = X$

Bound Laws:

- $X \cup U = U$        $X \cap \phi = \phi$

Absorption Laws:

- $X \cup (X \cap Y) = X$        $X \cap (X \cup Y) = X$

De Morgan's Laws:

- $(X \cup Y)' = X' \cap Y'$        $(X \cap Y)' = X' \cup Y'$

### 3.3. Relation and Function

A *relation* is an association between two sets of information. Consider a set of residents of a city and their phone numbers. The pairing of names with corresponding phone numbers is a relation. This pairing is *ordered* for the entire relation. For each pair, either the name comes first, followed by the phone number, or the reverse. The set from which the first element is drawn is called the *domain set*, and the other set is called the *range set*. The domain is what you start with, and the range is what you end with.

A function is a well-behaved relation. A relation  $R(X, Y)$  is well-behaved if every element of the domain set  $X$  corresponds to a single element of the range set  $Y$ . Consider domain set  $X$  as a set of people and range set  $Y$  as their phone numbers. If a person may have more than one phone number, then this

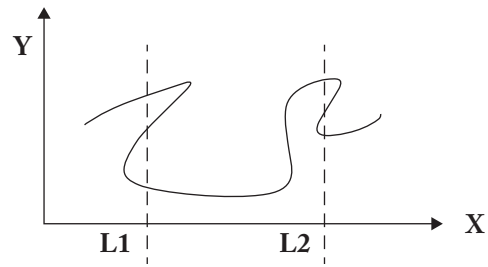


Figure 17.7. Vertical Line Test for Function

relation is not a function. However, if we draw a relation between the names of residents and their dates of birth with the name set as domain, then this becomes a well-behaved relation and hence a function. This means that while all functions are relations, not all relations are functions. In the case of a function given an  $x$ , there is one and exactly one  $y$  for each ordered pair  $(x, y)$ .

For example, consider the following two relations:

$$A: \{(3, -9), (5, 8), (7, -6), (3, 9), (6, 3)\}$$

$$B: \{(5, 8), (7, 8), (3, 8), (6, 8)\}$$

Are these functions as well?

In relation  $A$ , the domain is all  $x$ -values — i.e.,  $\{3, 5, 6, 7\}$  — and the range is all  $y$ -values — i.e.,  $\{-9, -6, 3, 8, 9\}$ .

Relation  $A$  is not a function, as there are two different range values,  $-9$  and  $9$ , for the same  $x$ -value,  $3$ .

In relation  $B$ , the domain is the same as for  $A$  — i.e.,  $\{3, 5, 6, 7\}$ . However, the range is a single element —  $\{8\}$ . This qualifies as a function even if all  $x$ -values are mapped to the same  $y$ -value. Here, each  $x$ -value is distinct, so the relation is well-behaved and is therefore a function. Therefore, Relation  $B$  may be represented by the equation  $y = 8$ .

Whether a relation may be characterized as a function can be verified using the vertical line test presented below:

Given the graph of a relation, if one can draw a vertical line that crosses the graph in more than one place, then that relation is not a function.



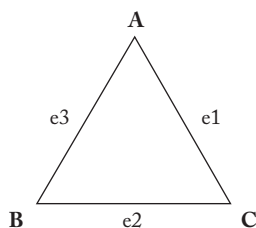


Figure 17.8. Example of a Graph

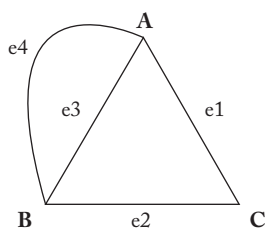


Figure 17.9. Example of a Multigraph

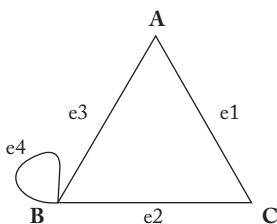


Figure 17.10. Example of a Pseudograph

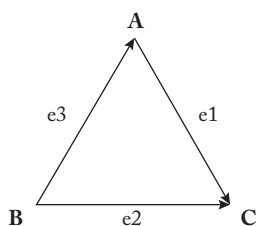


Figure 17.11. Example of a Directed Graph

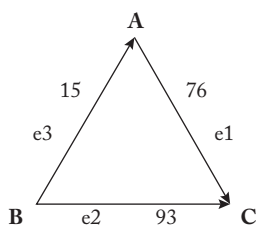


Figure 17.12. Example of a Weighted Graph

In Figure 17.7, both lines L1 and L2 cut the graph for the relation three times. This signifies that for each of these x-values (with L1 representing one x-value and L2 representing another), there are three different y-values. Thus, the relation is not a function. Of course, either L1 or L2 alone would be enough to prove that the relation is not a function.

## 4. Graph and Tree [1\*, c10, c11]

### 4.1. Graph

In a graph  $G = (V, E)$ ,  $V$  is the set of vertices (nodes) and  $E$  is the set of edges. Edges are also called *arcs* or *links*.

$F$  is a function that maps the set of edges  $E$  to a set of ordered or unordered pairs of elements  $V$ . In Figure 17.8,  $G = (V, E)$  where  $V = \{A, B, C\}$ ,  $E = \{e1, e2, e3\}$ , and  $F = \{(e1, (A, C)), (e2, (C, B)), (e3, (B, A))\}$ .

The simple graph in Figure 17.8 consists of a set of vertices or nodes and a set of edges connecting unordered pairs. The edges in simple graphs are undirected. Such graphs are also called *undirected graphs*. In Figure 17.8,  $(e1, (A, C))$  may be replaced by  $(e1, (C, A))$ , as the pair between vertices A and C is unordered. This is true for the other two edges as well.

In a multigraph, more than one edge may connect the same two vertices. Two or more connecting edges between the same pair of vertices may reflect multiple associations between the same two vertices. Such edges are called *parallel* or *multiple edges*. In Figure 17.9, the edges  $e3$  and  $e4$  both connect A and B. Figure 17.9 is a multigraph where edges  $e3$  and  $e4$  are multiple edges.

In a *pseudograph*, edges connecting a node to itself are allowed. Such edges are called *loops*.

In Figure 17.10, the edge  $e4$  both starts and ends at B. Figure 17.10 is a pseudograph in which  $e4$  is a loop.

A *directed graph*  $G = (V, E)$  consists of a set of vertices  $V$  and a set of edges  $E$  that are ordered pairs of elements of  $V$ . A directed graph may contain loops. In Figure 17.11,  $G = (V, E)$  is a directed graph where  $V = \{A, B, C\}$ ,



$E = \{e1, e2, e3\}$ , and  $F = \{(e1, (A, C)), (e2, (B, C)), (e3, (B, A))\}$ .

In *weighted graph*  $G = (V, E)$ , each edge has a weight associated with it. The weight of an edge typically represents the numeric value associated with the relationship between the corresponding two vertices. In Figure 17.12, the weights for the edges  $e1$ ,  $e2$  and  $e3$  are taken to be 76, 93 and 15, respectively. If the vertices A, B and C represent three cities in a state, the weights could be, for example, the distances in kilometers between these cities.

Let  $G = (V, E)$  be an undirected graph with edge set  $E$ . Then, for an edge  $e \in E$  where  $e = \{u, v\}$ , the following expressions are often used:

- $u, v$  are said to be *adjacent*, *neighbors*, or *connected*.
- Edge  $e$  is *incident* with vertices  $u$  and  $v$ .
- Edge  $e$  *connects*  $u$  and  $v$ .
- Vertices  $u$  and  $v$  are *endpoints* for edge  $e$ .

If vertex  $v \in V$ , in the undirected graph  $G = (V, E)$ , then:

- The *degree* of  $v$ ,  $\deg(v)$ , is its number of incident edges, except that any self-loops are counted twice.
- A vertex with degree 0 is called an *isolated vertex*.
- A vertex of degree 1 is called a *pendant vertex*.

Let  $G = (V, E)$  be a directed graph. If  $e(u, v)$  is an edge of  $G$ , then the following expressions can be used to describe the graph:

- $u$  is *adjacent to*  $v$ , and  $v$  is *adjacent from*  $u$ .
- $e$  *comes from*  $u$  and *goes to*  $v$ .
- $e$  *connects*  $u$  to  $v$ , or  $e$  *goes from*  $u$  to  $v$ .
- The *initial vertex* of  $e$  is  $u$ .
- The *terminal vertex* of  $e$  is  $v$ .

If vertex  $v$  is in the set of vertices for the directed graph  $G = (V, E)$ , then:

- *In-degree* of  $v$ ,  $\deg^-(v)$ , is the number of edges going to  $v$ , i.e., for which  $v$  is the terminal vertex.

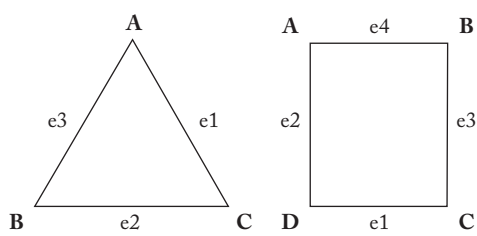


Figure 17.13. Example of Cycles  $C_3$  and  $C_4$

| Vertex | Adjacent Nodes |
|--------|----------------|
| A      | B, C           |
| B      | A, B, C        |
| C      | A, B           |

Figure 17.14. Adjacency List for the Graph in Figure 17.10

- *Out-degree* of  $v$ ,  $\deg^+(v)$ , is the number of edges coming from  $v$ , i.e., for which  $v$  is the initial vertex.
- *Degree* of  $v$ ,  $\deg(v) = \deg^-(v) + \deg^+(v)$ , is the sum of  $v$ 's in-degree and out-degree.
- A loop at a vertex contributes 1 to both the in-degree and the out-degree of this vertex.

According to the definitions above, the degree of a node is unchanged whether we consider its edges to be directed or undirected.

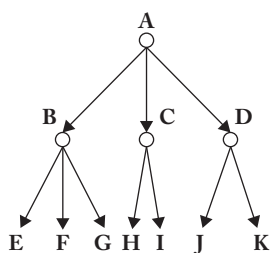
In an undirected graph, a path of length  $n$  from  $u$  to  $v$  is a sequence of  $n$  adjacent edges from vertex  $u$  to vertex  $v$ .

- A path is a *circuit* if  $u = v$ .
- A path *traverses* the vertices along it.
- A path is *simple* if it contains no edge more than once.

A cycle on  $n$  vertices  $C_n$  for any  $n \geq 3$  is a simple graph where  $V = \{v_1, v_2, \dots, v_n\}$  and  $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}, \{v_n, v_1\}\}$ .

For example, Figure 17.13 illustrates two cycles of lengths 3 and 4.

An *adjacency list* is a table with one row per vertex, listing its adjacent vertices. The



**Figure 17.15.** Example of a Tree

adjacency list for a directed graph maintains a listing of the terminal nodes for each vertex.

Figure 17.14 illustrates the adjacency lists for the pseudograph in Figure 17.10 and the directed graph in Figure 17.11. As the out-degree of vertex C in Figure 17.11 is 0, there is no entry against C in the adjacency list.

Different representations for a graph — e.g., adjacency matrix, incidence matrix and adjacency lists — need to be studied.

#### 4.2. Tree

A tree  $T(N, E)$  is a hierarchical data structure of  $n = |N|$  nodes with a specially designated root node  $R$  while the remaining  $n - 1$  nodes form subtrees under the root node  $R$ . The number of edges  $|E|$  in a tree are always equal to  $|N| - 1$ .

The subtree at node  $X$  is the subgraph of the tree consisting of node  $X$ , its descendants and all edges incident to those descendants. As an alternative to this recursive definition, a tree may be defined as a connected undirected graph with no simple circuits.

However, a tree is strictly hierarchical, whereas a graph is flat. In a tree, an ordered pair is built between two nodes as parent and child. Each child node in a tree is associated with only one parent node, whereas this restriction is meaningless for a graph, where no parent-child association exists.

An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Figure 17.15 presents a tree  $T(N, E)$  with a set of nodes  $N = \{A, B, C, D, E, F, G, H, I, J, K\}$ .

The edge set  $E$  is  $\{(A, B), (A, C), (A, D), (B, E), (B, F), (B, G), (C, H), (C, I), (D, J), (D, K)\}$ .

The parent of a non-root node  $v$  is the unique node  $u$  with a directed edge from  $u$  to  $v$ . Each node in the tree has a unique parent node except for the tree's root node. While root nodes can serve as parent nodes, they have no parent nodes themselves. In Figure 17.15, root node  $A$  is the parent node for nodes  $B$ ,  $C$  and  $D$ . Similarly,  $B$  is the parent of  $E$ ,  $F$  and  $G$ , and so on. The root node  $A$  has no parent.

A node that has children is called an *internal node*. For example, in Figure 17.15, node  $A$  and node  $B$  are examples of internal nodes.

The degree of a node in a tree is the same as its number of children. For example, in Figure 17.15, root node  $A$  and its child  $B$  are both of degree 3. Nodes  $C$  and  $D$  have degree 2.

A node's distance from the root node in number of hops is called its *level*. The root node is at level 0. Alternately, a node  $X$ 's level is the unique path's length from the tree's root to node  $X$ . Root node  $A$  is at level 0 in Figure 17.15. Nodes  $B$ ,  $C$  and  $D$  are at level 1. The remaining nodes in Figure 17.15 are at level 2.

A tree's height is the maximum of the levels of tree nodes. For example, in Figure 17.15, the tree's height is 2.

A node is called a *leaf* if it has no children, and the degree of a leaf node is 0. For example, in Figure 17.15, nodes  $E$  through  $K$  are leaf nodes with degree 0.

The ancestors or predecessors of a non-root node  $X$  are all the nodes in the path from the root to node  $X$ . For example, in Figure 17.15, nodes  $A$  and  $D$  form the set of ancestors for  $J$ .

A node  $X$ 's successors or descendants are all the nodes that have  $X$  as their ancestor. For a tree with  $n$  nodes, all remaining  $n - 1$  nodes are successors of the root node. In Figure 17.15, node  $B$  has successors in  $E$ ,  $F$ , and  $G$ .

*If node  $X$  is an ancestor of node  $Y$ , then node  $Y$  is a successor of  $X$ .*

Two or more nodes sharing the same parent node are called *sibling* nodes. For example, in Figure 17.15, nodes  $E$  and  $G$  are siblings. However, nodes  $E$  and  $J$ , though at the same level, are not sibling nodes.

Two sibling nodes are at the same level, but two nodes at the same level are not necessarily siblings.

A tree is called an *ordered tree* if the relative position of occurrences of children nodes is significant. For example, a family tree is an ordered tree if, as a rule, the name of an elder sibling always appears before (on the left of) the younger sibling.

In an unordered tree, the relative position of occurrences between the siblings has no significance and may be altered arbitrarily.

A binary tree is formed with 1 or more nodes where there is a root node  $R$  and all the remaining nodes form a pair of ordered subtrees under the root node. In a binary tree, no internal node can have more than two children. In addition to this criterion for the degree of internal nodes, a binary tree is always ordered. If the positions of the left and right subtrees for any node in the tree are swapped, then a new tree is created.

In Figure 17.16, the two binary trees are different, as the positions of occurrences of  $A$ 's children differ in the two trees.

According to [1\*], a binary tree is called a *full binary tree* if every internal node has exactly two children. For example, the binary tree in Figure 17.17 is a full binary tree, as both internal nodes  $A$  and  $B$  are of degree 2.

A full binary tree that meets the definition above is also called a *strictly binary tree*.

Both binary trees in Figure 17.18 are complete binary trees. The tree in Figure 17.18(a) is a complete and full binary tree. A complete binary tree has all its levels filled, except possibly the last one. If a complete binary tree's last level is not full, nodes occur from the left-most positions available.

Interestingly, following the definitions above, the tree in Figure 17.18(b) is a complete but not full binary tree, as node  $B$  has only one child in  $D$ . On the contrary, the tree in Figure 17.17 is a full but not complete binary tree, as  $B$ 's children occur in the tree, whereas the children of  $C$  do not appear in the last level.

A binary tree of height  $H$  is balanced if all leaf nodes occur at levels  $H$  or  $H - 1$ . All three binary trees in Figures 17.17 and 17.18 are balanced binary trees.

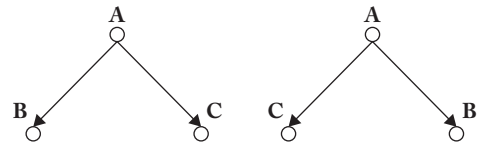


Figure 17.16. Examples of Binary Trees

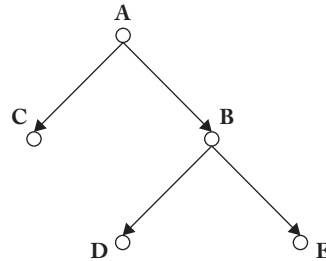


Figure 17.17. Example of a Full Binary Tree

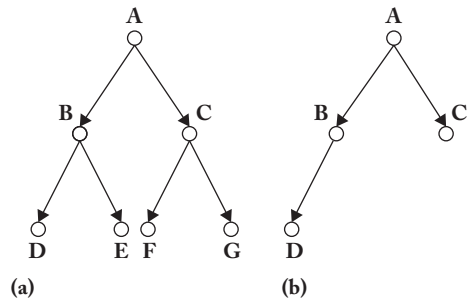


Figure 17.18. Example of Complete Binary Trees

There are at most  $2^H$  leaves in a binary tree of height  $H$ . In other words, if a binary tree with  $L$  leaves is full and balanced, then its height is  $H = \lceil \log_2 L \rceil$ . This is true for the two trees in Figures 17.17 and 17.18(a), as both trees are full and balanced. However, the tree in Figure 17.18(b) is not a full binary tree.

A *binary search tree (BST)* is a special kind of binary tree in which each node contains a distinct key value, and the key value of each node in the tree is less than every key value in its right subtree and greater than every key value in its left subtree.

A *traversal algorithm* is a procedure for systematically visiting every node of a binary tree. Tree traversals may be defined recursively.

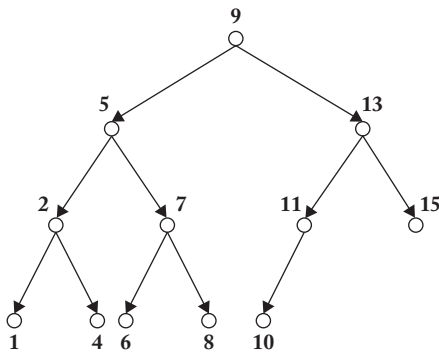


Figure 17.19. A Binary Search Tree

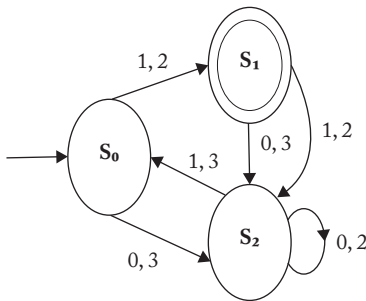


Figure 17.20. Example of an FSM

If  $T$  is a binary tree with root  $R$  and the remaining nodes form an ordered pair of non-null left subtree  $T_L$  and nonnull right subtree  $T_R$  below  $R$ , then the preorder traversal function  $\text{PreOrder}(T)$  is defined as:

$$\begin{aligned} \text{PreOrder}(T) &= R, \text{PreOrder}(T_L), \\ &\text{PreOrder}(T_R) \end{aligned}$$

eqn. 1

The recursive process of finding the preorder traversal of the subtrees continues until the subtrees are found to be null. Here, commas have been used as delimiters for improved readability.

The postorder and in-order may be similarly defined using eqn. 2 and eqn. 3, respectively.

$$\begin{aligned} \text{PostOrder}(T) &= \text{PostOrder}(T_L), \\ &\text{PostOrder}(T_R), R \end{aligned}$$

eqn. 2

$$\begin{aligned} \text{InOrder}(T) &= \text{InOrder}(T_L), R, \text{InOrder}(T_R) \\ &\text{eqn. 3} \end{aligned}$$

The tree in Figure 17.19 is a binary search tree (BST). The pre-order, post-order and in-order traversal outputs for this BST are given below in their respective orders:

Preorder output: 9, 5, 2, 1, 4, 7, 6, 8, 13, 11, 10, 15

Postorder output: 1, 4, 2, 6, 8, 7, 5, 10, 11, 15, 13, 9

In-order output: 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 13, 15

## 5. Finite-State Machine

[1\*, c13]

A computer system may be abstracted as a mapping from state to state, driven by inputs. In other words, a system may be considered a transition function  $T: S \times I \rightarrow S \times O$ , where  $S$  is the set of states and  $I$  and  $O$  are the input and output functions.

If the state set  $S$  is finite, the system is called a *finite-state machine* (FSM).

Alternatively, a finite state machine (FSM) is a mathematical abstraction composed of a finite number of states and transitions between those states. For example, if the domain  $S \times I$  is reasonably small, then one can specify  $T$  explicitly, using diagrams similar to a flow graph to illustrate how logic flows for different inputs. However, this is practical only for machines with a very small information capacity.

An FSM has a finite internal memory, an input feature that reads symbols one at a time in a sequence, and an output feature.

The operation of an FSM begins from a start state, goes through transitions depending on the input to different states, and can end in any valid state. However, only a few of the states mark a successful flow of operation. These are called *accept states*.

The information capacity of an FSM is  $C = \log |S|$ . Thus, if we represent a machine having an information capacity of  $C$  bits as an FSM, then its state transition graph will have  $|S| = 2^C$  nodes.

An FSM is formally defined as  $M = (S, I, O, f, g, s_0)$ .

$S$  is the state set.

$I$  is the set of input symbols.

$O$  is the set of output symbols.

$f$  is the state transition function.

$g$  is the output function.

$s_0$  is the initial state.

Given an input  $x \in I$  on state  $S_k$ , the FSM transitions to state  $S_j$ , following state transition function  $f$ , and produces an output  $y \in O$ , using the output function  $g$ .

Figure 17.20 illustrates an FSM with  $S_0$  as the start state and  $S_1$  as the final state. Here,  $S = \{S_0, S_1, S_2\}$ ;  $I = \{0, 1\}$ ;  $O = \{2, 3\}$ ;  $f(S_0, 0) = S_2$ ;  $f(S_0, 1) = S_1$ ;  $f(S_1, 0) = S_2$ ;  $f(S_1, 1) = S_2$ ;  $f(S_2, 0) = S_2$ ;  $f(S_2, 1) = S_0$ ;  $g(S_0, 0) = 3$ ;  $g(S_0, 1) = 2$ ;  $g(S_1, 0) = 3$ ;  $g(S_1, 1) = 2$ ;  $g(S_2, 0) = 2$ ;  $g(S_2, 1) = 3$ .

The state transition and output values for different inputs on different states may instead be represented using a state table. The state table for the FSM in Figure 17.20 is shown in Figure 17.21. Each pair against an input symbol represents the new state and the output symbol. Figures 17.21(a) and 17.21(b) are alternative representations of the FSM in Figure 17.20.

## 6. Grammar

[1\*, c13]

The grammar of a natural language defines whether a combination of words makes a valid sentence. Unlike natural languages, the syntax of a formal language is specified by a well-defined set of rules. The valid sentences of a formal language can be described by a grammar with the help of these rules, called *production rules*.

A formal language is a set of finite-length words or strings over some finite alphabet, and a grammar specifies the rules for forming those words or strings. The entire set of words that are valid for a grammar constitutes the language for the grammar. Thus, the grammar  $G$  is any compact, precise mathematical definition of a language  $L$  as opposed to a raw listing of all legal sentences or examples of those sentences in that language.

| Current State | Input      |   |
|---------------|------------|---|
|               | 0          | 1 |
| $S_0$         | $S_2, S_1$ |   |
| $S_1$         | $S_2, S_2$ |   |
| $S_2$         | $S_2, S_0$ |   |

(a)

| Current State | Output |   | State |       |
|---------------|--------|---|-------|-------|
|               | Input  |   | Input |       |
|               | 0      | 1 | 0     | 1     |
| $S_0$         | 3      | 2 | $S_2$ | $S_1$ |
| $S_1$         | 3      | 2 | $S_2$ | $S_2$ |
| $S_2$         | 2      | 3 | $S_2$ | $S_0$ |

(b)

Figure 17.21. Tabular Representation of an FSM

A grammar implies an algorithm that can generate all legal sentences of the language. There are different types of grammars.

A phrase structure grammar (PSG) or Type-0 grammar  $G = (V, T, S, P)$  is a 4-tuple in which:

- $V$  is the vocabulary — i.e., the set of words.
- $T \subseteq V$  is a set of words called *terminals*.
- $S \in V$  is a special word called the *start symbol*.
- $P$  is the set of production rules for substituting one sentence fragment for another.

There exists another set,  $N = V - T$ , of words called *nonterminals*. The nonterminals represent concepts such as *noun*. Production rules are applied on strings containing nonterminals until no more nonterminal symbols are present in the string. The start symbol  $S$  is a nonterminal.

The language generated by a formal grammar  $G$ , denoted by  $L(G)$ , is the set of all strings over the set  $V$  that can be generated, starting with the start symbol, by applying production rules until all the nonterminal symbols are replaced in the string.

For example, let  $G = (\{S, A, a, b\}, \{a, b\}, S, \{S \rightarrow aA, S \rightarrow b, A \rightarrow aa\})$ . Here, the set of terminals is  $N = \{S, A\}$ , where  $S$  is the start symbol. The three production rules for the grammar are given as  $P1: S \rightarrow aA$ ;  $P2: S \rightarrow b$ ;  $P3: A \rightarrow aa$ .

Applying the production rules in all possible ways, the following words may be generated from the start symbol:

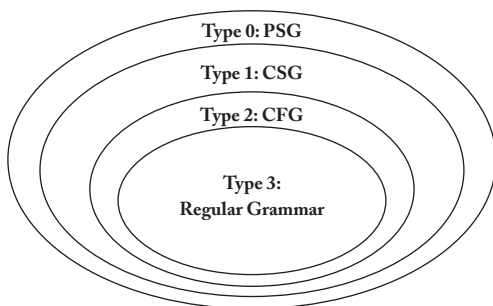


Figure 17.22. Chomsky Hierarchy of Grammars

$S \rightarrow aA$  (using  $P1$  on start symbol)  
 $\rightarrow aaa$  (using  $P3$ )  
 $S \rightarrow b$  (using  $P2$  on start symbol)

Nothing else can be derived from  $G$ . Thus, the language of the grammar  $G$  consists of only two words:  $L(G) = \{aaa, b\}$ .

### 6.1. Language Recognition

Formal grammars can be classified according to the types of productions they allow. The Chomsky hierarchy (introduced by Noam Chomsky in 1956) describes such a classification scheme.

From Figure 17.22, we can infer the following about different grammars:

1. Every regular grammar is a context-free grammar (CFG).
2. Every CFG is a context-sensitive grammar (CSG).
3. Every CSG is a phrase structure grammar (PSG).

**Context-Sensitive Grammar (CSG):** All fragments in the RHS are either longer than the corresponding fragments in the LHS or empty; in other words, if  $b \rightarrow a$ , then  $|b| < |a|$  or  $a = \emptyset$ . A formal language is context-sensitive if a CSG generates it.

**Context-Free Grammar (CFG):** All fragments in the LHS are of length 1; in other words, if  $A \rightarrow a$ , then  $|A| = 1$  for all  $A \in N$ . The term context-free derives from the fact

that  $A$  can always be replaced by  $a$ , regardless of the context in which it occurs. A formal language is context-free if a CFG generates it. Context-free languages are the theoretical basis for the syntax of most programming languages.

**Regular Grammar:** All fragments in the RHS are either single terminals or a pair built by a terminal and a nonterminal; if  $A \rightarrow a$ , then either  $a \in T$ ,  $a = cD$ , or  $a = Dc$  for  $c \in T$ ,  $D \in N$ .

If  $a = cD$ , the grammar is called a *right linear grammar*. On the other hand, if  $a = Dc$ , the grammar is called a *left linear grammar*. Both the right linear and left linear grammars are regular or Type-3 grammars.

The language  $L(G)$  generated by a regular grammar  $G$  is called a *regular language*.

A regular expression  $A$  is a string (or pattern) formed from the following pieces of information:  $a \in \Sigma$ , the set of alphabets,  $\epsilon$ , 0, and the operations OR (+), PRODUCT ( $\cdot$ ), and CONCATENATION (\*). The language of  $G$ ,  $L(G)$  is equal to all those strings that match  $G$ ,  $L(G) = \{x \in \Sigma^* | x \text{ matches } G\}$ .

For any  $a \in \Sigma$ ,  $L(a) = a$ ;  $L(\epsilon) = \{\epsilon\}$ ;  $L(0) = \emptyset$ .

- + functions as an or,  $L(A + B) = L(A) \cup L(B)$ .
- creates a product structure,  $L(AB) = L(A) \cdot L(B)$ .
- \* denotes concatenation,  $L(A^*) = \{x_1 x_2 \dots x_n \mid x_i \in L(A) \text{ and } n \geq 0\}$ .

For example, the regular expression  $(ab)^*$  matches the set of strings:  $\{\epsilon, ab, abab, ababab, abababab, \dots\}$ . The regular expression  $(aa)^*$  matches the set of strings on one letter 'a' with even length. The regular expression  $(aaa)^* + (aaaaa)^*$  matches the set of strings of length equal to a multiple of 3 or 5.

## 7. Number Theory [1\*, c4]

Number theory is one of the oldest branches of pure mathematics and one of the largest. It concerns questions about numbers, usually meaning whole numbers, and fractional or



rational numbers. The different types of numbers include, for example, whole, integer, rational, real, and complex.

### 7.1. Types of Numbers

*Natural Numbers:* This group of numbers starts at 1 and continues with 2, 3, 4, 5 and so on. Zero is not in this group. There are no negative or fractional numbers in the group of natural numbers. The common mathematical symbol for the set of all natural numbers is  $\mathbb{N}$ .

*Whole Numbers:* This group has all natural numbers plus the number 0.

Unfortunately, not everyone accepts the above definitions of natural and whole numbers. There is no general agreement about whether to include 0 in the set of natural numbers. Many mathematicians consider that, in Europe, the sequence of natural numbers traditionally started with 1 (0 was not even considered a number by the Greeks). In the 19th century, set theoreticians and other mathematicians started the convention of including 0 in the set of natural numbers.

*Integers:* This group includes all the whole numbers and their negatives. The common mathematical symbol for the set of all integers is  $\mathbb{Z}$  — i.e.,  $\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$ .

*Rational Numbers:* These numbers can be expressed as a ratio of two integers. The common symbol for the set of all rational numbers is  $\mathbb{Q}$ .

Rational numbers may be classified into three types based on how the decimals act: (1) decimals do not exist (e.g., 15); or (2) decimals do exist, and they terminate (e.g., 15.6); (3) decimals do exist, and they repeat with a pattern, as in 1.666... (which is  $5/3$ ).

*Irrational Numbers:* These numbers cannot be expressed as an integer divided by an integer. These numbers have decimals that never terminate and never repeat with a pattern (e.g.,  $\pi$  or  $\sqrt{2}$ ).

*Real Numbers:* This group comprises all rational and irrational numbers. The numbers algebra uses are real numbers. The common mathematical symbol for the set of all real numbers is  $\mathbb{R}$ .

*Imaginary Numbers:* These are all based on the imaginary number  $i$ . This imaginary number is equal to the square root of  $-1$ . Any real number multiple of  $i$  is an imaginary number (e.g.,  $i$ ,  $5i$ ,  $3.2i$ ,  $-2.6i$ ).

*Complex Numbers:* A complex number is a combination of a real number and an imaginary number in the form  $a + bi$ . The real part is  $a$ , and  $b$  is called the imaginary part. The common mathematical symbol for the set of all complex numbers is  $\mathbb{C}$ . For example,  $2 + 3i$ ,  $3 - 5i$ ,  $7.3 + 0i$ , and  $0 + 5i$  are complex numbers, but the latter two are equivalent to real numbers.  $7.3 + 0i$  is the same as the real number 7.3. Similarly,  $0 + 5i$  is same as the imaginary number  $5i$ . All real numbers are complex numbers with 0 for the imaginary part, and all imaginary numbers are complex numbers with 0 for the real part.

### 7.2. Divisibility

Elementary number theory involves divisibility among integers. Let  $a, b \in \mathbb{Z}$  with  $a \neq 0$ . The expression  $a|b$  says that  $a$  divides  $b$  if  $\exists c \in \mathbb{Z}$ , and the expression  $b = ac$  means that there is an integer  $c$  such that  $c$  times  $a$  equals  $b$ . For example,  $3|-12$  is true, but  $3|7$  is false.

If  $a$  divides  $b$ , then we say that  $a$  is a factor of  $b$  or  $a$  is a divisor of  $b$ , and  $b$  is a multiple of  $a$ .  $b$  is even if and only if  $2 | b$ .

Let  $a, d \in \mathbb{Z}$  with  $d > 0$ . Then  $a \bmod d$  denotes the remainder  $r$  from the division algorithm with dividend  $a$  and divisor  $d$ , i.e., the remainder when  $a$  is divided by  $d$ . We can compute  $(a \bmod d)$  by  $a - d \lfloor a/d \rfloor$ , where  $\lfloor a/d \rfloor$  represents the floor of the real number.

Let  $\mathbb{Z}^+ = \{n \in \mathbb{Z} \mid n > 0\}$  and  $a, b \in \mathbb{Z}$ ,  $m \in \mathbb{Z}^+$ . Then  $a$  is congruent to  $b$  modulo  $m$ , written as  $a \equiv b \pmod{m}$ , if and only if  $m | a - b$ .

Alternately,  $a$  is congruent to  $b$  modulo  $m$  if and only if  $(a - b) \bmod m = 0$ .

### 7.3. Prime Number

An integer  $p > 1$  is prime if and only if it is not the product of any two integers greater than 1; i.e.,  $p$  is prime if  $p > 1 \wedge \neg \exists a, b \in \mathbb{N}: a > 1, b > 1, a * b = p$ .



The only positive factors of a prime  $p$  are 1 and  $p$  itself. The numbers 2, 13, 29, 61, etc., are prime numbers. Nonprime integers greater than 1 are called *composite numbers*. A composite number may be composed by multiplying two integers greater than 1.

There are many interesting applications of prime numbers; among them is the public-key cryptography scheme, which involves the exchange of public keys containing the product  $p \cdot q$  of two random large primes  $p$  and  $q$  (a private key) that must be kept secret by a given party.

#### 7.4. Greatest Common Divisor

The greatest common divisor  $\gcd(a, b)$  of integers  $a, b$  is the greatest integer  $d$  that is a divisor both of  $a$  and of  $b$  — i.e.,

$$d = \gcd(a, b) \text{ for } \max(d: d|a \wedge d|b).$$

For example,  $\gcd(24, 36) = 12$ .

Integers  $a$  and  $b$  are called *relatively prime* or *coprime* if and only if  $\gcd(a, b) = 1$ . For example, neither 35 nor 6 is prime, but they are coprime because  $\gcd(35, 6) = 1$ .

A set of integers  $X = \{i_1, i_2, \dots\}$  is relatively prime if all possible pairs  $i_b, i_k, b \neq k$  drawn from the set  $X$  are relatively prime.

### 8. Basics of Counting [1\*, c6]

The *sum rule* states that if a task  $t_1$  can be done in  $n_1$  ways and a second task  $t_2$  can be done in  $n_2$  ways, and if these tasks cannot be done at the same time, then there are  $n_1 + n_2$  ways to do either task.

- If  $A$  and  $B$  are disjoint sets, then  $|A \cup B| = |A| + |B|$ .
- In general if  $A_1, A_2, \dots, A_n$  are disjoint sets, then  $|A_1 \cup A_2 \cup \dots \cup A_n| = |A_1| + |A_2| + \dots + |A_n|$ .

If 200 athletes do sprint events and 30 other athletes participate in the long jump event, then how many ways are there to

pick one athlete who is either a sprinter or a long jumper?

Using the sum rule, the answer would be  $200 + 30 = 230$ .

The *product rule* states that if a task  $t_1$  can be done in  $n_1$  ways and a second task  $t_2$  can be done in  $n_2$  ways after the first task has been done, then there are  $n_1 \cdot n_2$  ways to perform the procedure.

- If  $A$  and  $B$  are disjoint sets, then  $|A \times B| = |A| \cdot |B|$ .
- In general, if  $A_1, A_2, \dots, A_n$  are disjoint sets, then  $|A_1 \times A_2 \times \dots \times A_n| = |A_1| \cdot |A_2| \cdot \dots \cdot |A_n|$ .

If 200 athletes do sprint events and 30 other athletes participate in the long jump event, then how many ways are there to pick two athletes so that one is a sprinter and the other is a long jumper?

Using the product rule, the answer would be  $200 \cdot 30 = 6,000$ .

The *principle of inclusion-exclusion* states that if a task  $t_1$  can be done in  $n_1$  ways and a second task  $t_2$  can be done in  $n_2$  ways at the same time with  $t_1$ , then to find the total number of ways the two tasks can be done, one must subtract the number of ways to do both tasks from  $n_1 + n_2$ .

- If  $A$  and  $B$  are not disjoint,  $|A \cup B| = |A| + |B| - |A \cap B|$ .

In other words, the principle of inclusion-exclusion aims to ensure that the objects in the intersection of two sets are not counted more than once.

*Recursion* is the general term for defining an object in terms of itself. There are recursive algorithms, recursively defined functions, relations, sets, etc.

A recursive function is a function that calls itself. For example, we can define  $f(n) = 3 \cdot f(n - 1)$  for all  $n \in \mathbb{N}$  and  $n \neq 0$  and  $f(0) = 5$ .

An algorithm is recursive if it solves a problem by reducing it to an instance of the same problem with a smaller input.

A phenomenon is said to be *random* if individual outcomes are uncertain but the

long-term pattern of many individual outcomes is predictable.

The probability of any outcome for a random phenomenon is the proportion of times the outcome would occur in a very long series of repetitions.

The probability  $P(A)$  of any event  $A$  satisfies  $0 \leq P(A) \leq 1$ . Any probability is a number between 0 and 1. If  $S$  is the sample space in a probability model, then  $P(S) = 1$ . All possible outcomes together must have a probability of 1.

Two events are disjoint if they have no outcomes in common and so can never occur together. If  $A$  and  $B$  are two disjoint events,  $P(A \text{ or } B) = P(A) + P(B)$ . This is known as the *addition rule for disjoint events*.

If two events have no outcomes in common, the probability that one or the other occurs is the sum of their individual probabilities.

*Permutation* is an arrangement of objects in which the order matters without repetition. For example, one can choose  $r$  objects in a particular order from a total of  $n$  objects by using  ${}^nP_r$  ways, where  ${}^nP_r = n! / (n - r)!$ . Various notations, such as  ${}^nP_r$  and  $P(n, r)$ , are used to represent the number of permutations of a set of  $n$  objects taken  $r$  at a time.

*Combination* is a selection of objects in which the order does not matter without repetition. This is different from a permutation because the order does not matter. If only the order is changed (and not the members), no new combination is formed. One can choose  $r$  objects in any order from a total of  $n$  objects using  ${}^nC_r$  ways, where  ${}^nC_r = n! / [r! * (n - r)!]$ .

## 9. Discrete Probability [1\*, c7]

*Probability* is the mathematical description of randomness. Basic definitions of probability and randomness are provided in the previous section. Here, we start with the concepts behind probability distribution and discrete probability.

A *probability model* is a mathematical description of a random phenomenon consisting of two parts: a sample space  $S$  and a way of assigning probabilities to events. The

sample space defines the set of all possible outcomes, whereas an event is a subset of a sample space representing a possible outcome or a set of outcomes.

A *random variable* is a function or rule that assigns a number to each outcome. Basically, it is a symbol that represents the outcome of an experiment. For example,  $X$  could be the number of heads when the experiment is flipping a coin  $n$  times. Similarly,  $S$  could be the speed of a passing car as measured on a radar detector.

The values for a random variable could be discrete or continuous, depending on the experiment. A discrete random variable can hold all possible values (i.e., can represent all possible outcomes) without missing any, although it might take an infinite amount of time. A *continuous random variable* is used to measure an uncountable number of values even when an infinite amount of time is given.

For example, if random variable  $X$  represents an outcome that is a real number between 1 and 100, then  $X$  may have an infinite number of values. Therefore, one can never list all possible outcomes for  $X$ , even if an infinite amount of time is allowed. Here,  $X$  is a continuous random variable. On the other hand, for the same interval of 1 to 100, another random variable  $Y$  can be used to list all integer values in the range. Here,  $Y$  is a discrete random variable.

An uppercase letter, say  $X$ , will represent the *name* of the random variable. Its lowercase counterpart,  $x$ , will represent the *value* of the random variable.

The probability that the random variable  $X$  will equal  $x$  is:

$P(X = x)$  or, more simply,  $P(x)$ .

A Probability Distribution (Density) Function (PDF) is a table, formula or graph that describes the values of a random variable and the probabilities associated with these values. Probabilities associated with discrete random variables have the following properties:

| $X$    | 1   | 2   | 3   | 4   | 5   | 6   |
|--------|-----|-----|-----|-----|-----|-----|
| $P(x)$ | 1/6 | 1/6 | 1/6 | 1/6 | 1/6 | 1/6 |

Figure 17.23. A Discrete Probability Function for Rolling a Die

- $0 \leq P(x) \leq 1$  for all  $x$
- $\Sigma P(x) = 1$

A *discrete probability distribution* can be represented as a discrete random variable.

The *mean*  $\mu$  of a *probability distribution model* is the sum of the product terms for individual events and their outcome probability. In other words, for the possible outcomes  $x_1, x_2, \dots, x_n$  in a sample space  $S$  if  $p_k$  is the probability of outcome  $x_k$ , the mean of this probability would be  $\mu = x_1 p_1 + x_2 p_2 + \dots + x_n p_n$ . The mean of the probability density function for the distribution in Figure 17.23 would be the following:

$$1 * (1/6) + 2 * (1/6) + 3 * (1/6) + 4 * (1/6) + 5 * (1/6) + 6 * (1/6) \\ = 21 * (1/6) = 3.5$$

Here, the sample space refers to the set of all possible outcomes.

The *variance*  $\sigma^2$  of a *discrete probability model* is  $\sigma^2 = (x_1 - \mu)^2 p_1 + (x_2 - \mu)^2 p_2 + \dots + (x_k - \mu)^2 p_k$ . The *standard deviation*,  $\sigma$ , is the square root of the variance. For the probability distribution in Figure 17.23, the variation  $\sigma^2$  would be the following:

$$\sigma^2 = [(1 - 3.5)^2 * (1/6) + (2 - 3.5)^2 * (1/6) \\ + (3 - 3.5)^2 * (1/6) + (4 - 3.5)^2 * (1/6) + (5 - 3.5)^2 * (1/6) + (6 - 3.5)^2 * (1/6)] \\ = (6.25 + 2.25 + 0.25 + 0.5 + 2.25 + 6.25) * (1/6) \\ = 17.5 * (1/6) \\ = 2.90$$

$\therefore$  standard deviation  $s = 1.70$

These numbers aim to derive the average value from repeated experiments. This is based

on the most important principle in probability — i.e., the average value from repeated experiments is likely to be close to the expected value of one experiment. Moreover, the average value is more likely to be closer to the expected value of any one experiment as the number of experiments increases.

## 10. Numerical Precision, Accuracy, and Error [2\*, c1]

The main goal of numerical analysis is to develop efficient algorithms for computing precise numerical values of functions, as well as finding solutions to algebraic and differential equations, optimization problems, etc.

Digital computers can store finite numbers only. A digital computer cannot represent any infinitely large number — be it an integer, rational number, or any real or complex number [see section 7, Number Theory]. The mathematics of approximation is critical for working with numbers in the finite range a computer can handle.

Each number in a computer is assigned a location (e.g., an address or register) and consists of a quantity of binary digits, or bits. A  $k$ -bit location can store any of  $N = 2^k$  different numbers. A 32-bit location can store any of  $N = 2^{32} \approx 4.3 \times 10^9$  different numbers, while a 64-bit location can store any of  $N = 2^{64} \approx 1.84 \times 10^{19}$  different numbers. The question is how to distribute those numbers for maximum efficiency and accuracy in practical computations.

One choice is to distribute the numbers evenly, leading to fixed-point arithmetic. In this system, the first bit represents the sign, and the remaining bits represent magnitude. The decimal point — more appropriately, the binary point (the transition between whole and fractional values) — can be anywhere. Integer numbers are represented by placing the binary point immediately to the right of the least significant bit, and integer numbers between  $-2^{k-1}-1$  and  $2^{k-1}$  can be stored. Placing the binary point to the left of the least significant bit allows non-integer values to be represented.

Another choice is to space the numbers closely together, say with a uniform gap of  $2^{-n}$ , and thereby distribute the total  $N$  numbers uniformly over the interval  $-2^{-n-1}N < x \leq 2^{-n-1}N$ . Real numbers lying between the gaps are represented by either *rounding* (meaning the closest exact representative) or *chopping* (meaning the exact representative immediately below — or above, if negative — the number).

Numbers outside the range must be represented by the largest (or largest negative) number that can be represented. This becomes a symbol for overflow, which occurs when a computation produces a value outside the range.

When processing speed is a significant bottleneck, fixed-point representations can be an attractive and faster alternative to the more cumbersome floating-point arithmetic most commonly used in practice.

Accuracy and precision are important terms in numerical analysis.

*Accuracy* is the closeness with which a measured or computed value agrees with the true value.

*Precision*, on the other hand, is the closeness with which two or more measured or computed values for the same thing agree. In other words, precision is the closeness with which a number represents an exact value.

Let  $x$  be a real number, and let  $x^*$  be an approximation. The *absolute error* in the approximation  $x^* \approx x$  is defined as  $|x^* - x|$ . The *relative error* is defined as the ratio of the absolute error to the size of  $x$  — i.e.,  $|x^* - x| / |x|$  — which assumes  $x \neq 0$ ; otherwise, relative error is not defined. For example, 1,000,000 is an approximation of 1,000,001 with an absolute error of 1 and a relative error of  $10^{-6}$ , whereas 10 is an approximation of 11 with an absolute error of 1 and a relative error of 0.1. Typically, relative error is more intuitive and the preferred determiner of the size of the error. The present convention is that errors are always  $\geq 0$  and are  $= 0$  if and only if the approximation is exact.

An approximation  $x^*$  has  $k$  significant decimal digits if its relative error is  $< 5 \times 10^{-k-1}$ . This means that the first  $k$  digits of  $x^*$  following its first nonzero digit are the same as those of  $x$ .

Significant digits are the digits of a number that are known to be correct. In a measurement, one uncertain digit is included. For example, measurement of length with a ruler of 15.5 mm with  $\pm 0.5$  mm maximum allowable error has two significant digits, whereas a measurement of the same length using a caliper and recorded as 15.47 mm with  $\pm 0.01$  mm maximum allowable error has three significant digits.

## 11. Algebraic Structures

This section introduces a few representations used in higher algebra. An algebraic structure consists of one or two sets closed under some operations and satisfying several axioms, including none. Group, monoid, ring and lattice are examples of algebraic structures. Group, monoid and ring are defined in this section.

### 11.1. Group

A set  $S$  closed under a binary operation  $\bullet$  forms a group if the binary operation satisfies the following four criteria:

- **Associative:**  $\forall a, b, c \in S$ , the equation  $(a \bullet b) \bullet c = a \bullet (b \bullet c)$  holds.
- **Identity:** There exists an identity element  $I \in S$  such that for all  $a \in S$ ,  $I \bullet a = a \bullet I = a$ .
- **Inverse:** Every element  $a \in S$  has an inverse  $a' \in S$  with respect to the binary operation, i.e.,  $a \bullet a' = I$ ; for example, the set of integers  $\mathbb{Z}$  with respect to the addition operation is a group. The identity element of the set is 0 for the addition operation. In  $\forall x \in \mathbb{Z}$ , the inverse of  $x$  would be  $-x$ , which is also included in  $\mathbb{Z}$ .
- **Closure property:**  $\forall a, b \in S$ , the result of the operation  $a \bullet b \in S$ .

A group that is commutative i.e.,  $a \bullet b = b \bullet a$  is known as a *commutative* or *Abelian group*.

The set of natural numbers  $\mathbb{N}$  (with the operation of addition) is not a group because there is no inverse for any  $x > 0$  in the set of natural numbers. (The third criterion, the

inverse criterion, is violated.) However, the set of natural numbers has some structure.

Sets with an associative operation (the first criterion) are called *semigroups*; if they also have an identity element (the second criterion), they are called *monoids*. The set of natural numbers under addition is an example of a monoid, a structure that is not quite a group because it is missing the requirement that every element have an inverse under the operation.

A monoid is a set  $S$  that is closed under a single associative binary operation  $\cdot$  and has an identity element  $I \in S$  such that for all  $a \in S$ ,  $I \cdot a = a \cdot I = a$ . A monoid must contain at least one element. The set of natural numbers  $\mathbb{N}$  forms a commutative monoid under addition with identity element 0. The same set of natural numbers  $\mathbb{N}$  also forms a monoid under multiplication with identity element 1. The set of positive integers  $\mathbb{P}$  forms a commutative monoid under multiplication with identity element 1.

It may be noted that, unlike those in a group, elements of a monoid need not have inverses. A monoid can also be considered a semigroup with an identity element.

A *subgroup* is a group  $H$  contained within a bigger group,  $G$ , such that the identity element of  $G$  is contained in  $H$ , and whenever  $b_1$  and  $b_2$  are contained in  $H$ , so are  $b_1 \cdot b_2$  and  $b_1^{-1}$ . Thus, the elements of  $H$ , equipped with the group operation on  $G$  restricted to  $H$ , form a group.

Given any subset  $S$  of a group  $G$ , the subgroup generated by  $S$  consists of products of elements of  $S$  and their inverses. It is the smallest subgroup of  $G$  containing  $S$ . For example, let  $G$  be the Abelian group whose elements are  $G = \{0, 2, 4, 6, 1, 3, 5, 7\}$  and whose group operation is addition modulo 8. This group has a pair of nontrivial subgroups:  $J = \{0, 4\}$  and  $H = \{0, 2, 4, 6\}$ , where  $J$  is also a subgroup of  $H$ .

In group theory, a *cyclic group* is a group that can be generated by a single element, in the sense that the group has an element  $a$  (called the *generator* of the group) such that, when this element is written multiplicatively, every element of the group is a power of  $a$ .

A group  $G$  is cyclic if  $G = \{a^n \text{ for any integer } n\}$ .

Since any group generated by an element in a group is a subgroup of that group, showing that the only subgroup of a group  $G$  that contains  $a$  is  $G$  itself suffices to show that  $G$  is cyclic. For example, the group  $G = \{0, 2, 4, 6, 1, 3, 5, 7\}$ , with respect to addition modulo 8 operation, is cyclic. The subgroups  $J = \{0, 4\}$  and  $H = \{0, 2, 4, 6\}$  are also cyclic.

## 11.2. Ring

If we take an Abelian group and define a second operation on it, a new structure is found that is different from just a group. If this second operation is associative and is distributive over the first, then we have a ring.

A *ring* is a triple of the form  $(S, +, \cdot)$ , where  $(S, +)$  is an Abelian group,  $(S, \cdot)$  is a semigroup and  $\cdot$  is distributive over  $+$ ; i.e.,  $\forall a, b, c \in S$ , the equation  $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$  holds. Further, if  $\cdot$  is commutative, then the ring is said to be commutative. If there is an identity element for the  $\cdot$  operation, then the ring is said to have an identity.

As an example,  $(\mathbb{Z}, +, *)$ , i.e., the set of integers  $\mathbb{Z}$  with the usual addition and multiplication operations, is a ring. As  $(\mathbb{Z}, *)$  is commutative, this ring is a commutative or Abelian ring. The ring has 1 as its identity element.

Note that the second operation may not have an identity element, nor do we need to find an inverse for every element with respect to this second operation. As for what distributive means, intuitively, it is what we do in elementary mathematics when we perform the following operation:  $a * (b + c) = (a * b) + (a * c)$ .

A *field* is a ring for which the elements of the set, excluding 0, form an Abelian group with the second operation. A simple example of a field is the field of rational numbers  $(\mathbb{Q}, +, *)$  with the usual addition and multiplication operations. The numbers are of the form  $a/b \in \mathbb{R}$ , where  $a, b$  are integers and  $b \neq 0$ . The additive inverse of such a fraction is simply  $-a/b$ , and the multiplicative inverse is  $b/a$ , provided that  $a \neq 0$ .

## 12. Engineering Calculus

Calculus is a branch of mathematics that deals with study of continuous transition, derivatives and integrals of functions using methods originally based on the summation of infinitesimal differences. Engineering Calculus focuses on learning analytical geometry and vectors for engineering applications.

Engineering Calculus includes the following:

- Limits
- Continuity
- Differentiation
- Integration
- Transcendental functions
- Vector calculus

Limits are the building blocks of calculus. For a function  $f(x)$ , the limit of the function at a point  $a$  is the value the function achieves at that point.

$$L = \lim_{x \rightarrow a} f(x)$$

A function is said to be *Continuous* on the interval  $[a, b]$  if it is continuous at each point in the interval.

$$\lim_{x \rightarrow a} f(x) = f(a)$$

The two major elements of calculus are differential calculus and integral calculus.

- *Differential calculus* analyzes the rate of change of one quantity in relation to the rate of change of another. Geometrically, it is the slope of the line tangent to the graph of the function. The rate of change of  $y$  with respect to  $x$  is expressed as  $dy/dx$ .
- *Integral calculus* analyzes such concepts as the area or volume enclosed by a function.

A transcendental function, in contrast to an algebraic function, is an analytic function that does not satisfy a polynomial equation.

Vector calculus deals with the differentiation and integration of vector fields in the three-dimensional Euclidean space.

Software engineers are encouraged to learn Engineering Calculus with case studies. These concepts are required for analyzing and extrapolating data.

## 13. New Advancements

### 13.1. Computational Neurosciences

Computational Neurosciences is a branch of Neurosciences that uses mathematical models, computer simulations and brain abstraction to understand and analyze cognitive abilities of the nervous systems. This enables the learning of control theory, cybernetics, quantitative psychology, machine learning, artificial intelligence, creativity, imagination, and connectionism among others.

The central assumption of computational neuroscience is that the brain computes. Generally speaking, a computer is a dynamic system whose state variables encode information about the external world. In short, computation equals coding plus dynamics. Some neuroscientists study the way that information is encoded in neural activity and other dynamic variables of the brain. Others try to characterize how these dynamic variables evolve over time. The study of neural dynamics can be further subdivided into two separate strands. One tradition, exemplified by the work of Hodgkin and Huxley, focuses on the biophysics of single neurons. The other focuses on the dynamics of networks, concerning itself with phenomena that emerge from the interactions between neurons. Therefore computational neuroscience can be divided into three sub-specialties: neural coding, biophysics of neurons, and neural networks.

### 13.2. Genomics

The in-silico analysis of nucleotide sequences of chromosome(s) from a given



organism is called “genome”. The genome is the genetic material of living organisms, containing hereditary characteristics. It is constituted by DNA. Genomic studies aim to understand how genes and genetic information are organized within the genome and how this organization determines their function.

Genomics deals with structure, function, mapping, evolution and editing of genomes, including sequencing and analysis of genomes.

Significant research works are being undertaken in the areas of preventive and therapeutic healthcare, especially in the area of detection, analysis and repair of genetic disorders. These include genome data security, genome data sharing, efficiency in genome data analysis among others.

Genomics encompasses a variety of techniques and approaches, including DNA sequencing, bioinformatic analysis, study of genetic variation, computational modeling, and much more.

The advancement of DNA sequencing technologies and bioinformatic analysis has significantly propelled progress in genomics, enabling detailed study of genomes across various organisms.

Due to the large amount of data represented by nucleotide sequences obtained from genome sequencing, informatics is required to handle these data. And the development of specific software for the field relies heavily on Software Engineering.

## MATRIX OF TOPICS VS. REFERENCE MATERIAL

|                                                    | Rosen 2018 [1*] | Cheney and Kincaid 2020 [2*] |
|----------------------------------------------------|-----------------|------------------------------|
| <b>1. Basic Logic</b>                              | c1              |                              |
| <b>2. Proof Techniques</b>                         | c1              |                              |
| <b>3. Set, Relation, Function</b>                  | c2              |                              |
| <b>4. Graph and Tree</b>                           | c10, c11        |                              |
| <b>5. Finite State Machine</b>                     | c13             |                              |
| <b>6. Grammar</b>                                  | c13             |                              |
| <b>7. Number Theory</b>                            | c4              |                              |
| <b>8. Basics of Counting</b>                       | c6              |                              |
| <b>9. Discrete Probability</b>                     | c7              |                              |
| <b>10. Numerical Precision, Accuracy and Error</b> |                 | c1                           |
| <b>11. Algebraic Structures</b>                    |                 |                              |
| <b>12. Calculus</b>                                |                 |                              |

## REFERENCES

- [1\*] K. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., McGraw-Hill, 2018.
- [2\*] E.W. Cheney and D.R. Kincaid, *Numerical Mathematics and Computing*, 7th ed., Addison Wesley, 2020.